

“深入多线程内部机理”之四



# 并发编程三特性（下）

---

北京理工大学计算机学院  
金旭亮

# 有序性



有序性 (Ordering) 指在什么情况下一个处理器上运行的一个线程所执行的多个内存访问操作，其顺序与在另外一个处理器上运行的其他线程所看到的操作顺序是一致的，比如，处理器一上执行的访问内存指令序列为“修改A” --> “修改B” --> “修改C”，则处理器二上另一个线程所感知到的内存变量值的改变序列也是“A变” --> “B变” --> “C变”，两者是一致的。

“有序性”之所以会成为问题，是因为只要满足特定的规则或条件，为了提升性能，编译器和处理器都有可能在不影响单线程程序正确性的前提下，对指令进行“重排序”。但这种重排序，有可能会对多线程程序的正确性产生影响。

# JIT编译器重排序示例

在Java程序运行时，JIT (Just-In-Time Compiler, 即时编译器) 可以动态地改变程序代码运行的顺序：



这样做的主要原因是CPU流水线是同时执行这 4个指令的，那么轻耗时的代码在很大程度上可以先执行完，从而可以让出CPU流水线资源给其他指令，所以代码重排序是为了追求更高的程序运行效率。

# “数据依赖性”对排序有着关键性的影响

如果两个操作访问同一个变量，且这两个操作中有一个为写操作，此时这两个操作之间就存在数据依赖性。

| 名 称 | 代码示例                         | 说 明            |
|-----|------------------------------|----------------|
| 写后读 | <pre>a = 1;<br/>b = a;</pre> | 写一个变量之后，再读这个位置 |
| 写后写 | <pre>a = 1;<br/>a = 2;</pre> | 写一个变量之后，再写这个变量 |
| 读后写 | <pre>a = b;<br/>b = 1;</pre> | 读一个变量之后，再写这个变量 |

编译器和处理器在重排序时，会尊重数据间存在的这种依赖性，不会改变存在数据依赖关系的两个操作的执行顺序。

# as-if-serial语义不能被违背



在计算机中，软件技术和硬件技术有一个共同的目标：在不改变程序执行结果的前提下，尽可能提高并行度，但不管怎样，编译器、runtime和处理器都必须遵循as-if-serial语义。



as-if-serial语义的意思是：不管怎么重排序，（单线程）程序的执行结果不能被改变。

# 禁止重排序



为了遵循as-if-serial语义，保证多线程程序的正确性，在必要时，我们可以使用关键字volatile禁止代码重排序：

A变量的操作  
B变量的操作  
volatile z变量的操作  
C变量的操作  
D变量的操作



- 1) A、 B可以重排序。
- 2) C、 D可以重排序。
- 3) A、 B不可以重排到Z的后面。
- 4) C、 D不可以重排到Z的前面。

换言之，变量Z是一个“屏障”， Z变量之前或之后的代码不可以跨越它，这就是屏障的作用。



# 内存屏障指令



禁止重排序，主要通过调用处理器提供的“内存屏障”（Memory Barriers, Intel称之为Memory Fence）指令实现。

指令块1



指令块2



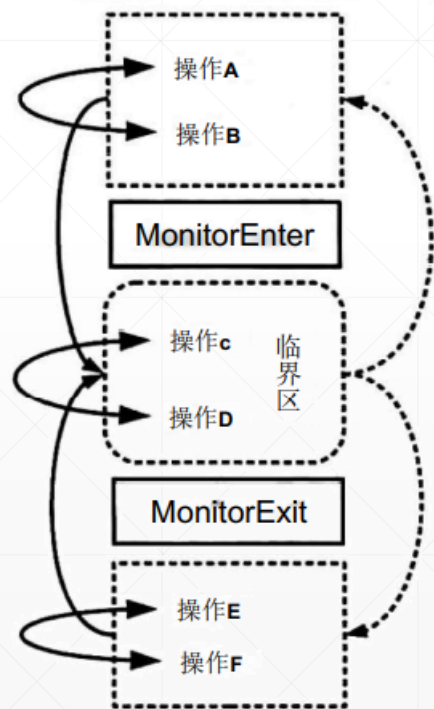
指令块3



它在指令序列中就像是一堵墙一样使其两侧（之前和之后）的指令无法“穿越”它，即阻止了“跨越墙”的排序。

# “许进不许出”的临界区

借助于处理器的屏障指令，Java实现了临界区的“许进不许出”特性。



临界区内、临界区前和临界区后这3个区域内的任意两个操作都可以在各自的区域范围内进行重排序（只要相应的重排序能够满足as-if-serial语义）。

临界区外的语句可以被（编译器）重排序到临界区之内（“许进”），而临界区内的操作无法被（编译器或者处理器）重排到临界区之外（“不许出”）。



# 可见性 vs. 有序性



可见性是有序性的基础。可见性描述的是一个线程对共享变量的更新对于另外一个线程是否可见，或者说什么情况下可见的问题。



有序性描述的是，一个处理器上运行线程，它对共享变量的更新，对于另外一个在其他处理器运行的线程看来，这些操作是按照什么顺序执行的问题。



有序性影响可见性。由于重排序的作用，一个线程对共享变量的更新，对于另外一个线程而言，可能变得不可见。